

2025 - 2026

PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

SPÉCIALITÉ : INFORMATIQUE

**SUJET : Conception et développement d'une
solution de transfert interbancaire hybride**

Réalisé par: Melki Mohamed Aziz

Encadrant ESPRIT: Mm Malak Hedhli

Encadrant Entreprise: Mm Farah Khlif



Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous :

- Nom & Prénom: Melki Mohamed Aziz

Encadrant Entreprise

- Nom & Prénom: Mm Farah Khlif

Cachet & Signature



Encadrant Académique

- Nom & Prénom : Malek Hedhli

Signature / Signature



Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma sincère reconnaissance envers toutes les personnes qui, par leur soutien, leurs conseils et leur disponibilité, ont contribué à la réussite de ce travail.

Je souhaite tout d'abord remercier chaleureusement mon encadrante académique, Madame Malek Hedhli, pour sa disponibilité constante, la pertinence de ses conseils et son accompagnement rigoureux tout au long de ce projet. Son suivi attentif et ses remarques constructives m'ont permis de structurer ma démarche, d'affiner la problématique et de progresser sereinement dans les différentes étapes de ce travail.

Je tiens également à adresser mes remerciements les plus sincères à mon encadrante en entreprise, Madame Farah Khlif, pour la confiance qu'elle m'a accordée dès le début du stage, pour son encadrement quotidien et pour le temps précieux qu'elle a consacré à répondre à mes questions et à valider mes choix techniques. Son expertise et sa bienveillance ont grandement facilité mon intégration au sein de l'équipe et la bonne conduite du projet.

Mes remerciements s'adressent aussi à Monsieur Klai Ghassen, expert académique, dont les retours avisés et les échanges enrichissants ont permis d'améliorer la qualité scientifique et technique de ce rapport, ainsi que la pertinence des choix d'architecture retenus.

Je remercie par ailleurs l'ensemble de l'équipe qui m'a accompagné durant ce projet de fin d'études, pour son esprit collaboratif, sa réactivité lors des phases de test et de correction des anomalies, ainsi que pour l'ambiance de travail stimulante qui a rendu cette expérience aussi formatrice qu'agréable.

Enfin, je remercie ma famille et mes proches pour leur soutien moral tout au long de ce stage, ainsi que toutes les personnes qui, de près ou de loin, ont contribué à l'aboutissement de ce projet.

Résumé

Ce rapport présente le travail réalisé dans le cadre d'un projet de fin d'études portant sur la conception et le développement d'une plateforme de paiement transfrontalier nommée Cross-Border Payment. Ce projet répond à une problématique concrète du secteur bancaire : comment automatiser, sécuriser et rendre traçables les transferts interbancaires internationaux, tout en garantissant la conformité réglementaire et une expérience utilisateur professionnelle.

La solution proposée repose sur une architecture hybride à trois couches, combinant les forces complémentaires de plusieurs technologies. Le frontend, développé avec Angular en mode standalone et mis en forme avec Bootstrap 5, offre une interface claire et ergonomique aux employés de banque ainsi qu'aux responsables de la Banque Centrale. Le backend, construit avec Node.js, Express et PostgreSQL, centralise la logique métier, la persistance des données et la sécurité applicative via une authentification JWT. Enfin, un smart contract Solidity, déployé sur un réseau Ethereum local géré avec Hardhat, constitue le cœur de confiance du système : il encadre les règles de transfert, la gestion des banques, la blacklist AML et l'historisation des transactions de manière immuable et vérifiable.

Le projet a été conduit selon une démarche agile inspirée de la méthodologie Scrum, organisée en sprints itératifs correspondant chacun à un périmètre fonctionnel cohérent : gestion du smart contract, authentification et gestion des banques, transferts interbancaires, conformité et pilotage, puis reporting et industrialisation. Cette organisation a permis de livrer progressivement des incréments fonctionnels démontrables, tout en maintenant une cohérence globale entre les trois couches de l'application.

Parmi les fonctionnalités clés développées figurent la gestion dynamique des taux de change synchronisés avec une API externe, le calcul automatique des frais de transfert, la validation humaine des transferts sensibles dépassant un seuil défini, l'administration d'une liste noire conforme aux exigences AML, un tableau de bord d'indicateurs clés de performance, ainsi que l'export de rapports au format PDF. La solution a également été containerisée avec Docker afin de faciliter son déploiement et sa reproductibilité.

Ce travail démontre la faisabilité et la pertinence d'une architecture combinant blockchain et services traditionnels pour répondre aux enjeux de sécurité, de transparence et de conformité propres au secteur des paiements interbancaires internationaux.

Abstract

This report presents the development of CrossBorder Payment, a platform designed to automate and secure cross-border interbank payments. The system relies on a three-layer architecture combining an Angular frontend, a Node.js/Express backend with a PostgreSQL database, and a Solidity smart contract deployed on a local Ethereum network via Hardhat. The research addresses a central question: how can a hybrid architecture ensure transaction security, regulatory compliance (notably anti-money laundering), and a user experience tailored to financial institutions? The project followed an agile sprint-based methodology, delivering successive functional modules including smart contract deployment, partner bank management, transfer processing, compliance tools, and Docker-based industrialisation. Key results show that the platform successfully ensures transaction integrity and traceability through the immutable ledger, while the backend orchestrates business logic, exchange rate synchronisation, and reporting. The findings demonstrate that combining traditional banking reliability with blockchain transparency offers a relevant and operational response to the limitations of existing cross-border payment solutions, paving the way for future enhancements in multi-currency support and reinforced authentication.

Contents

Remerciements	3
Résumé	4
Abstract	5
Introduction Générale	11
Présentation du Projet	11
Contexte Organisationnel	11
Contexte Technique	11
Problématique	11
Objectifs du Stage	11
Méthodologie de Travail	12
Organisation Quotidienne	12
Cadre Théorique	12
Structure du Rapport	12
1 Contexte du Projet	13
1.1 Étude de Marché	13
1.1.1 Contexte du Marché des Paiements Transfrontaliers	13
1.1.2 Analyse des Solutions Existantes	13
1.1.3 Positionnement du Projet	14
1.1.4 Public Cible	14
1.2 Étude d'Ingénierie des Exigences	14
1.2.1 Identification des Acteurs	14
1.2.2 Recueil des Besoins Fonctionnels	14
1.2.3 Recueil des Besoins Non Fonctionnels	14
1.2.4 Priorisation des Exigences	14
1.2.5 Spécification des Exigences	15
1.2.6 Contraintes du Projet	15
1.3 Conclusion du Chapitre	15
2 Sprint 0 - Analyse Conceptuelle et Méthodologie	16
2.1 Modèles d'Analyse	16
2.1.1 Diagrammes UML	16
2.2 Méthodologie Agile	18
2.2.1 Scrum	18
2.2.2 Rôles de l'Équipe Scrum	18
2.2.3 Cérémonies Scrum Adoptées	19
2.2.4 Product Backlog	19
2.2.5 Planification des Sprints	19
2.3 Conclusion	19

3	Sprint 1 — Smart Contracts	21
3.1	Sprint Backlog	21
3.2	Architecture Blockchain en Couches	21
3.3	Structure du Smart Contract	21
3.4	Modèle de Données Blockchain	23
3.5	Gestion des Permissions	23
3.6	Flux de Transfert dans le Smart Contract	23
3.7	Traitement des Transferts Volumineux	24
3.8	Exécution Interne et Conversion	24
3.9	Taux de Change et Frais	24
3.10	Blacklist AML	24
3.11	Événements et Audit	24
3.12	Tests et Validation	24
3.13	Limites et Améliorations Possibles	25
3.14	Conclusion	25
4	Sprint 2 — Onboarding et Gestion des Banques	26
4.1	Sprint Backlog	26
4.2	Authentification et Gestion des Rôles	26
4.2.1	Routes d'Authentification	27
4.2.2	Sécurisation de la Navigation	27
4.2.3	Écran de Connexion	27
4.2.4	Authentification en tant qu'Employé de Banque	27
4.2.5	Authentification en tant que Banque Centrale	28
4.3	Gestion des Banques	28
4.3.1	Intégration Blockchain	28
4.3.2	Écran de Gestion des Banques	28
4.4	Modèle de Données	29
4.5	Robustesse Applicative : Gestion des Erreurs et Stratégie de Tests	29
4.6	Conclusion	30
5	Sprint 3 — Transferts Interbancaires	31
5.1	Sprint Backlog	31
5.2	Parcours Utilisateur	31
5.3	Création de Transferts	32
5.4	Validation des Transferts en Attente	32
5.5	Exécution et Historisation	33
5.6	Traitement des Événements Blockchain	33
5.7	Modèle de Données	33
5.7.1	Exemples de Vues PostgreSQL	33
5.8	Fiabilisation du Parcours de Transfert : Gestion des Erreurs et Validation par les Tests	34
5.9	Conclusion	34

6	Sprint 4 — Conformité, Taux de Change et Pilotage	35
6.1	Sprint Backlog	35
6.2	Blacklist AML	35
6.2.1	Écran d'Administration de la Blacklist	36
6.3	Taux de Change et Frais.....	36
6.3.1	Automatisation	36
6.4	Tableau de Bord et Pilotage.....	37
6.4.1	Écran du Tableau de Bord	37
6.5	Modèle de Données.....	37
6.5.1	Exemples de Vues PostgreSQL.....	38
6.6	Notifications et Alertes.....	38
6.7	Tests et Validation	38
6.8	Conclusion	38
7	Sprint 5 — Reporting et Industrialisation	39
7.1	Sprint Backlog	39
7.2	Export PDF.....	39
7.3	Architecture Backend Consolidée.....	39
7.4	Base de Données PostgreSQL	40
7.5	Dockerisation.....	41
7.6	Industrialisation du Déploiement : Containerisation, Intégration Continue et Gestion des Risques	41
7.7	Difficultés Rencontrées	42
7.8	Conclusion	42
8	Évaluation et Perspectives	43
8.1	Analyse Critique	43
8.2	Bilan Personnel	43
8.3	Contributions du Projet	44
8.4	Perspectives d'Évolution.....	44
	Technologies Utilisées	46
	Conclusion Générale	47
	Bibliographie	48
	Glossaire des Abréviations	49

List of Figures

1	Comparaison des solutions de paiement transfrontalier	13
2	Diagramme de cas d'utilisation	16
3	Diagramme de classes	17
4	Diagramme de séquence système	18
5	Rôles de l'équipe Scrum.....	18
6	Diagramme de Gantt — Planification des sprints	19
7	Architecture blockchain en couches	21
8	Flux de transfert dans le smart contract	23
9	Résultats des tests Hardhat.....	25
10	Écran de connexion	27
11	Authentification employé de banque	28
12	Authentification Banque Centrale	28
13	Écran de gestion des banques.....	29
14	Schéma PostgreSQL — Banques	29
15	Formulaire de création de transfert.....	32
16	Validation des transferts en attente.....	32
17	Schéma PostgreSQL — Transferts.....	33
18	Administration de la blacklist AML	36
19	Gestion des taux de change	36
20	Tableau de bord de pilotage.....	37
21	Schéma PostgreSQL — Taux et blacklist	37
22	Notification par email	38
23	Export de rapports PDF	39
24	Architecture backend consolidée.....	40
25	Schéma complet de la base de données PostgreSQL	40
26	Conteneurs Docker.....	41

List of Tables

1	Priorisation des exigences selon la méthode MoSCoW.....	15
2	Product Backlog	20
3	Sprint Backlog — Smart Contract	22
4	Sprint Backlog — Onboarding et gestion des banques.....	26
5	Sprint Backlog — Transferts interbancaires.....	31
6	Sprint Backlog — Conformité, taux de change et pilotage.....	35
7	Sprint Backlog — Reporting et industrialisation	41

Introduction Générale

Ce chapitre introduit le contexte général du projet, la problématique à laquelle il répond, les objectifs poursuivis durant le stage ainsi que la méthodologie de travail adoptée. Il pose les bases nécessaires à la compréhension des chapitres suivants, qui détaillent successivement le contexte du projet puis l'analyse conceptuelle retenue.

Présentation du Projet

Le projet CrossBorder Payment se positionne comme une solution logicielle de paiement transfrontalier destinée aux institutions financières. Le système vise à automatiser des transferts interbancaires en combinant une interface web, une API métier et un smart contract blockchain. Il cible la réduction des risques, l'amélioration de la transparence et la facilitation de la supervision par la Banque Centrale.

Contexte Organisationnel

Ce stage s'est déroulé au sein d'UBCI, dans un service chargé d'innover sur les solutions de paiement. Le département est structuré autour de plusieurs équipes : développement, sécurité, produit et opérations. Le projet a été traité comme un cas d'étude réel, avec des livrables intermédiaires, des démonstrations et une documentation technique.

Contexte Technique

Le système repose sur une architecture hybride : un frontend Angular en standalone, un backend Node.js/Express, une base de données relationnelle PostgreSQL et un smart contract Solidity. Ce choix permet de tirer parti des forces de chaque technologie : réactivité de l'interface, robustesse du backend, persistance structurée et fiabilité du contrat blockchain.

Problématique

Comment concevoir une application de paiement interbancaire capable de garantir la sécurité, la conformité et l'automatisation des règles métier tout en offrant une expérience utilisateur claire et un suivi opérationnel efficace ?

Objectifs du Stage

Les objectifs du stage sont :

- Analyser les besoins métier et fonctionnels ;
- Définir une architecture technique cohérente ;
- Implémenter un smart contract pour sécuriser les transferts ;
- Développer un backend capable de centraliser les règles métier ;

- Concevoir un frontend ergonomique pour les utilisateurs ;
- Assurer la traçabilité, les alertes et l'export des rapports.

Méthodologie de Travail

La méthodologie adoptée est la méthode agile Scrum. Le projet est organisé en sprints itératifs, chacun comprenant une phase d'analyse et de spécification, une phase de conception, une phase de développement et une phase de validation. Des outils de suivi tels qu'un diagramme de Gantt, un product backlog et des réunions de revue de sprint ont été utilisés. Le détail de cette méthodologie, des rôles de l'équipe Scrum et de la planification des sprints est présenté au chapitre 3.

Organisation Quotidienne

La journée type était composée de réunions d'avancement (daily Scrum), de développement, de tests et de réunions de revue. Une attention particulière a été portée à la documentation des choix techniques et des évolutions du périmètre.

Cadre Théorique

Ce projet s'appuie sur plusieurs concepts théoriques : architecture n-tiers, principes REST, modèle relationnel, sécurité applicative basée sur JWT et rôles, gouvernance blockchain via smart contract, ainsi que les principes du cadre agile Scrum [3].

Structure du Rapport

Le rapport est structuré comme suit : le chapitre 1 présente le contexte du projet (étude de marché et étude d'ingénierie des exigences) ; le chapitre 2 détaille l'analyse conceptuelle (diagrammes UML, méthodologie Scrum, product backlog et planification des sprints) ; les chapitres suivants décrivent la réalisation technique du projet (smart contract, backend, frontend, fonctionnalités avancées), suivis d'une évaluation critique contenant les diagrammes, les extraits de code et la documentation technique.

1 Contexte du Projet

Ce chapitre présente le contexte dans lequel s'inscrit le projet CrossBorder Payment. Il détaille l'étude du marché des solutions de paiement transfrontalier ainsi que l'étude d'ingénierie des exigences ayant permis de définir le périmètre fonctionnel et non fonctionnel du système.

1.1 Étude de Marché

1.1.1 Contexte du Marché des Paiements Transfrontaliers

Le marché des paiements interbancaires internationaux est en pleine mutation. Les solutions traditionnelles, souvent basées sur le réseau SWIFT, présentent des délais de traitement variables, des frais élevés et un manque de transparence pour les institutions comme pour les clients finaux. L'émergence de technologies telles que la blockchain ouvre de nouvelles perspectives pour automatiser, sécuriser et rendre plus transparentes ces opérations [1].

1.1.2 Analyse des Solutions Existantes

- Les réseaux bancaires traditionnels (SWIFT) : fiables mais lents et coûteux, avec une traçabilité limitée pour les parties prenantes ;
- Les solutions de paiement blockchain publiques : rapides et transparentes, mais peu adaptées aux exigences réglementaires bancaires (AML, gouvernance centralisée) [2] ;
- Les plateformes fintech hybrides : combinent rapidité et conformité, mais restent peu répandues dans le contexte bancaire tunisien.

 SOLUTION	 DÉLAI	 COÛT	 FORCES	 FAIBLESSES
SWIFT	3–5 j	6–8 %	Universel	Lent, Cher
Western Union	<24 h	5–10 %	Mondial	Très cher
Wise	1–2 j	0.5–2 %	Transparent	Centralisé
Ripple	<5 s	~0 %	Rapide	Centralisé
 CrossBorder	<1 min	1 %	Décentralisé	Coûts gas

Figure 1: Comparaison des solutions de paiement transfrontalier

Ce tableau comparatif montre que SWIFT (3-5 jours, 6-8%) et Western Union (<24h, 5-10%) sont lents et coûteux, tandis que Wise (1-2 jours, 0.5-2%) et Ripple (<5s, 0%) sont plus rapides mais restent centralisés. CrossBorder Payment se distingue par sa rapidité (<1 minute), ses frais réduits (1%) et sa décentralisation, offrant une solution innovante et transparente pour les transferts internationaux.

1.1.3 Positionnement du Projet

CrossBorder Payment se positionne comme une solution hybride combinant la fiabilité d'une architecture bancaire classique (backend, base de données relationnelle, authentification) et la transparence apportée par un registre blockchain immuable. Cette approche répond directement aux limites identifiées dans les solutions existantes, tout en restant compatible avec les exigences de conformité imposées aux institutions financières telles qu'UBCI [5].

1.1.4 Public Cible

- Les employés de banques partenaires, qui initient et suivent les transferts ;
- Les responsables de la Banque Centrale, qui supervisent, valident et administrent le système ;
- À terme, les institutions financières souhaitant automatiser leurs échanges interbancaires transfrontaliers.

1.2 Étude d'Ingénierie des Exigences

1.2.1 Identification des Acteurs

- Employé de banque : consulte les soldes, initie des transferts, suit leur statut ;
- Banque Centrale : administre les banques, approuve les transferts en attente, gère la blacklist AML, consulte le tableau de bord.

1.2.2 Recueil des Besoins Fonctionnels

Gestion des banques et des comptes, consultation des taux de change, création et suivi des transferts, approbation des transferts en attente, administration de la blacklist, génération de rapports et export PDF.

1.2.3 Recueil des Besoins Non Fonctionnels

Temps de réponse inférieur à 2 secondes pour les pages critiques, sécurisation des données et des communications, maintenabilité du code, modularité et évolutivité, documentation et tests automatisés.

1.2.4 Priorisation des Exigences

Les exigences ont été priorisées selon la méthode MoSCoW (Must have, Should have, Could have, Won't have), afin de distinguer les fonctionnalités indispensables à la première version du produit de celles pouvant être différées à une itération ultérieure. Le tableau 1.1 synthétise cette priorisation.

Priorité	Exigences
Must have	Enregistrement des banques, création et exécution des transferts, blacklist AML
Should have	Approbation des transferts sensibles, tableau de bord des KPI
Could have	Export PDF, alertes sur les taux
Won't have (pour cette version)	Support multi-devises étendu, authentification multi-facteurs

Table 1: Priorisation des exigences selon la méthode MoSCoW

1.2.5 Spécification des Exigences

Chaque exigence fonctionnelle a été formalisée sous forme de user story, associée à des critères d'acceptation, afin de faciliter leur traduction en tâches de développement lors de la planification des sprints (voir chapitre 2).

1.2.6 Contraintes du Projet

- Contraintes réglementaires liées à la conformité AML [6] ;
- Contraintes techniques imposées par l'environnement bancaire existant [7] ;
- Contraintes de délai liées à la durée du stage ;
- Contraintes de sécurité propres au secteur financier.

1.3 Conclusion

Ce chapitre a permis de situer le projet CrossBorder Payment dans son contexte de marché et de formaliser les exigences fonctionnelles et non fonctionnelles à travers une démarche structurée d'ingénierie des exigences. Le chapitre suivant présente l'analyse conceptuelle du projet, à travers les diagrammes UML et la méthodologie Scrum adoptée.

2 Sprint 0 — Analyse Conceptuelle et Méthodologie

Ce chapitre a pour vocation de poser les fondements conceptuels et organisationnels du projet CrossBorder Payment, en offrant une vision claire et structurée des choix d'analyse et de méthode adoptés. Il s'articule autour de deux axes principaux : d'une part, la modélisation du système à travers les diagrammes UML, et d'autre part, la présentation de la méthodologie agile Scrum retenue pour la conduite du développement.

2.1 Modèles d'Analyse

2.1.1 Diagrammes UML

Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation permet d'identifier les interactions entre les acteurs du système (employé de banque, Banque Centrale) et les fonctionnalités offertes par la plateforme.

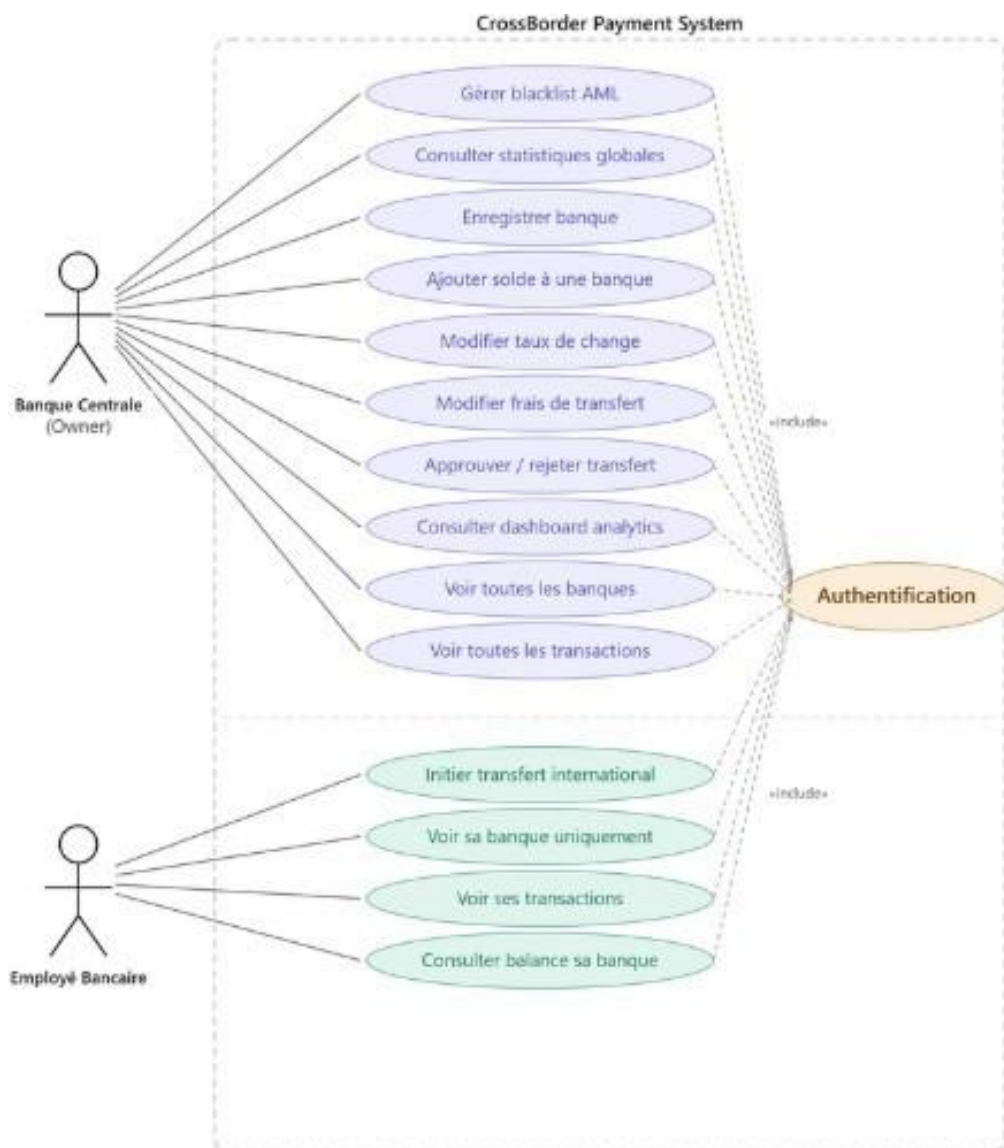


Figure 2: Diagramme de cas d'utilisation

Ce diagramme illustre les interactions entre les différents acteurs du système (Employé de banque, Administrateur de la Banque Centrale) et les principales fonctionnalités de la plateforme. Il met en évidence les cas d'utilisation clés tels que la gestion des transferts, l'administration des banques, la consultation des taux de change et la gestion de la conformité AML, définissant ainsi le périmètre fonctionnel du projet.

Diagramme de Classes

Le diagramme de classes modélise la structure statique du système : entités métier, attributs et relations entre les banques, les transferts et les utilisateurs.

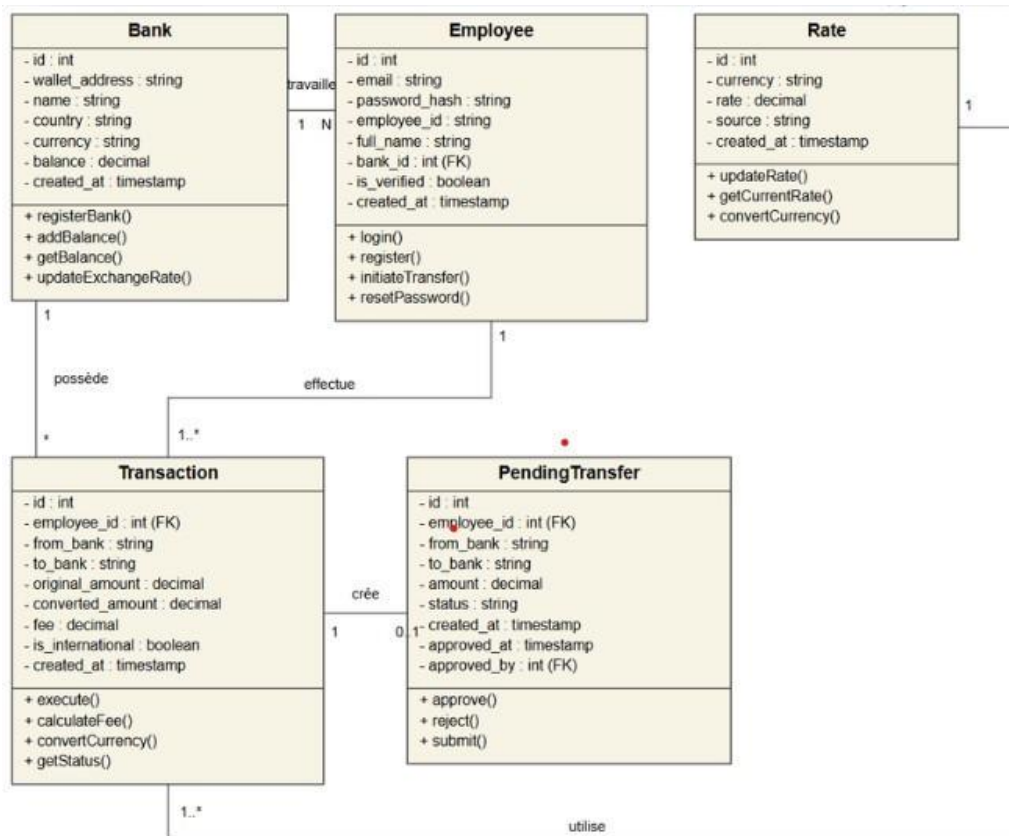


Figure 3: Diagramme de classes

Ce diagramme de classes modélise la structure statique du système backend. Il représente les principales entités métier telles que Bank, Transaction, PendingTransfer et Employee, ainsi que leurs attributs et les relations qui les lient, servant de base pour la conception de la base de données PostgreSQL [11] et de la logique métier.

Diagramme de Séquence Système

Le diagramme de séquence système illustre les échanges entre l'acteur et le système lors du scénario principal de création d'un transfert interbancaire, depuis la saisie jusqu'à la confirmation.

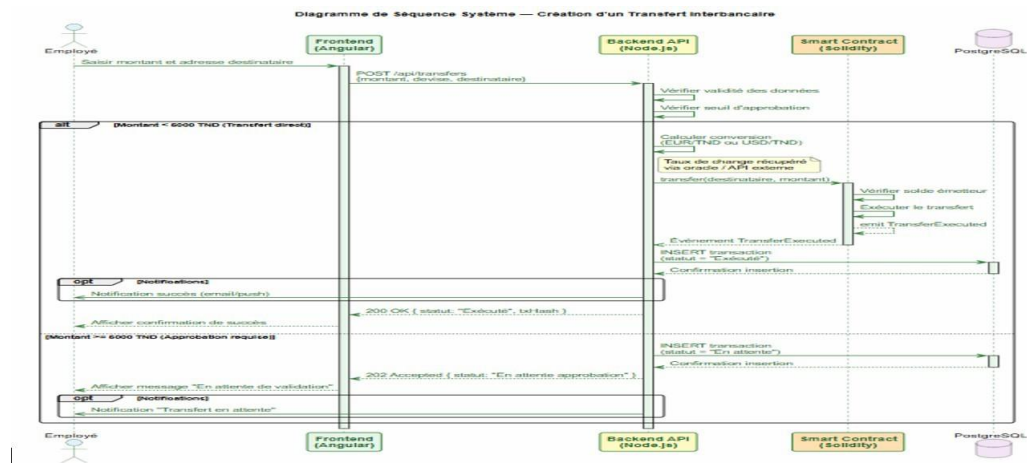


Figure 4: Diagramme de séquence système

2.2 Méthodologie Agile

2.2.1 Scrum

La méthodologie agile Scrum a été retenue pour la conduite de ce projet en raison de sa capacité à s'adapter à un périmètre évolutif, à favoriser des livraisons incrémentales et à faciliter la communication entre les parties prenantes [3]. Le découpage en sprints permet également de structurer le rapport de stage de façon cohérente avec l'avancement réel du projet.

2.2.2 Rôles de l'Équipe Scrum

- Product Owner : représente les intérêts métier, priorise le product backlog et valide les livrables de chaque sprint ;
- Scrum Master : facilite le déroulement des cérémonies Scrum, lève les obstacles rencontrés par l'équipe et veille au respect du cadre agile ;
- Équipe de développement : conçoit, développe et teste les incréments du produit à chaque sprint.

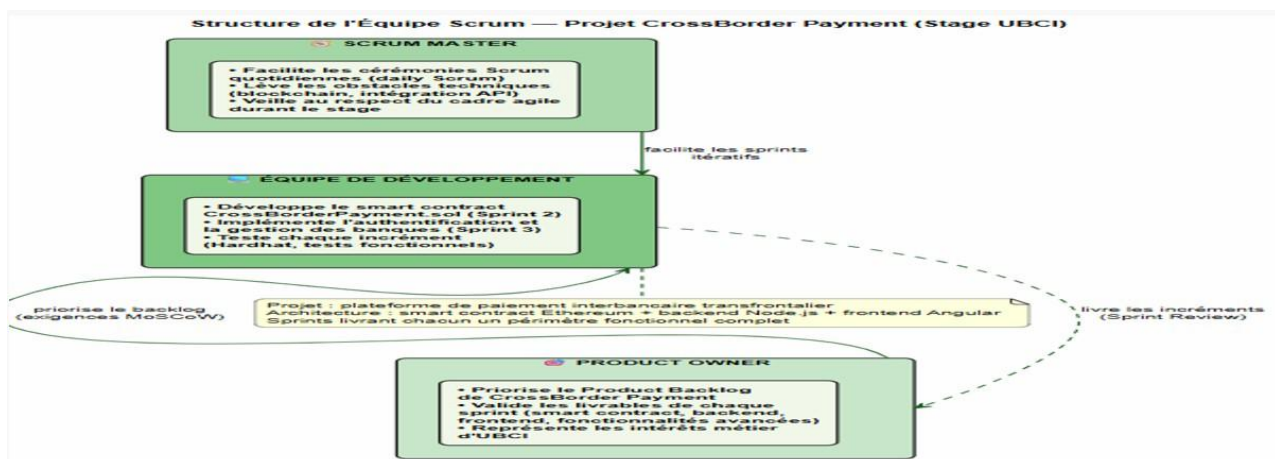


Figure 5: Rôles de l'équipe Scrum

Cette figure présente les 3 rôles réels du projet (Scrum Master, Équipe de développement, Product Owner) et leurs interactions. Chaque rôle reprend les livrables effectifs du stage (smart contract, authentification, backlog MoSCoW).

2.2.3 Cérémonies Scrum Adoptées

- Sprint Planning : définition des objectifs et sélection des éléments du backlog pour le sprint à venir ;
- Daily Scrum : point d'avancement quotidien et identification des blocages ;
- Sprint Review : démonstration de l'incrément réalisé ;
- Sprint Retrospective : analyse du déroulement du sprint et identification des axes d'amélioration.

2.2.4 Product Backlog

Le product backlog regroupe l'ensemble des exigences fonctionnelles formalisées sous forme de user stories, ordonnées par priorité métier. Le tableau 2.1 en présente le détail.

2.2.5 Planification des Sprints

Le projet a été découpé en sprints, chacun correspondant à un lot fonctionnel cohérent du système (smart contract, backend, frontend, fonctionnalités avancées). Cette planification a été formalisée à l'aide d'un diagramme de Gantt.

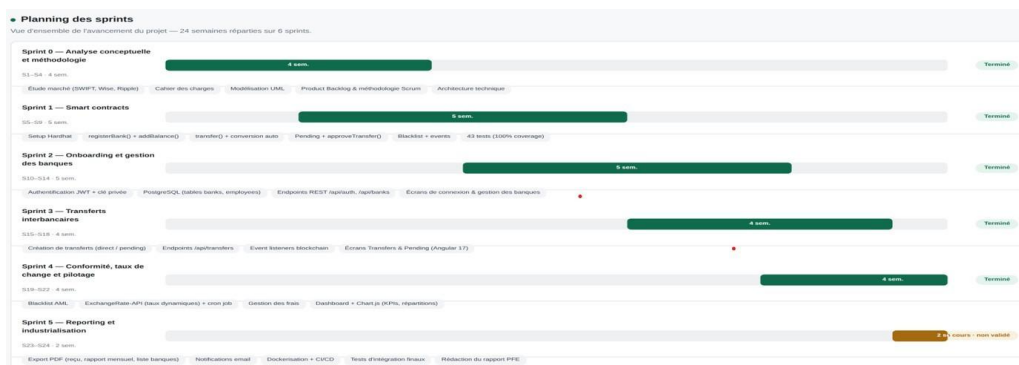


Figure 6: Diagramme de Gantt — Planification des sprints

2.3 Conclusion

Ce chapitre a permis de poser les bases conceptuelles du projet à travers la modélisation UML et de présenter le cadre méthodologique agile qui a guidé la conduite du projet. Les chapitres suivants détaillent la réalisation technique de chaque sprint, en commençant par la conception et l'implémentation du smart contract.

ID	Thème	User Story	Priorité	Attribuée à
US1	Gestion des banques	En tant que Banque Centrale, je veux enregistrer une nouvelle banque sur le réseau	P1	Banque Centrale
US2	Gestion des banques	En tant que Banque Centrale, je veux consulter la liste des banques enregistrées	P1	Banque Centrale
US3	Transferts	En tant qu'employé de banque, je veux créer un transfert vers une autre banque	P1	Banque Centrale
US4	Transferts	En tant qu'employé de banque, je veux suivre le statut de mes transferts	P1	Banque Centrale
US5	Taux de change	En tant que Banque Centrale, je veux mettre à jour les taux de change	P2	Banque Centrale
US6	Frais	En tant que Banque Centrale, je veux configurer les frais de transfert	P2	Banque Centrale
US7	AML	En tant que Banque Centrale, je veux gérer une blacklist AML	P1	Banque Centrale
US8	Authentification	En tant qu'utilisateur, je veux m'authentifier sur la plateforme	P1	Banque Centrale
US9	Tableau de bord	En tant que Banque Centrale, je veux consulter les KPI	P2	Banque Centrale
US10	Export PDF	En tant que Banque Centrale, je veux exporter des rapports PDF	P3	Banque Centrale
US11	Notifications	En tant qu'utilisateur, je veux recevoir des alertes par email	P3	Banque Centrale

Table 2: Product Backlog

3 Sprint 1 — Smart Contracts

Ce chapitre présente la conception et l'implémentation du smart contract `CrossBorderPayment.sol`, cœur de confiance de la plateforme. Le smart contract est développé en Solidity et déployé sur un réseau Ethereum local géré avec Hardhat [10].

3.1 Sprint Backlog

3.2 Architecture Blockchain en Couches

- Couche 1 : réseau Ethereum local Hardhat pour le développement et Sepolia pour les tests ;
- Couche 2 : Ethereum Virtual Machine (EVM) pour l'exécution du bytecode ;
- Couche 3 : smart contract `CrossBorderPayment.sol` implémentant la logique métier ;
- Couche 4 : application frontend Angular et backend Node.js communiquant via Ethers.js.



Figure 7: Architecture blockchain en couches

3.3 Structure du Smart Contract

- Gouvernance (propriétaire de la Banque Centrale) ;
- Registre des banques et des soldes ;
- Blacklist AML ;
- Taux de change et frais ;
- Moteur de transfert ;
- Gestion des limites quotidiennes ;
- État des transferts pendants ;
- Historique des transactions.

ID	Fonctionnalité	User Story	Priorité
SC1	Gouvernance	En tant que Banque Centrale, je veux être le propriétaire du contrat pour gérer les permissions	Must have
SC2	Registre des banques	En tant que Banque Centrale, je veux enregistrer les banques sur la blockchain	Must have
SC3	Gestion des soldes	En tant que Banque Centrale, je veux ajouter du solde aux banques enregistrées	Must have
SC4	Transferts inter-bancaires	En tant qu'employé de banque, je veux transférer des fonds vers une autre banque	Must have
SC5	Conversions de devises	En tant que Banque Centrale, je veux configurer les taux de change EUR/TND et USD/TND	Must have
SC6	Blacklist AML	En tant que Banque Centrale, je veux bloquer des banques suspectes	Must have
SC7	Transferts en attente	En tant que Banque Centrale, je veux approuver les transferts dépassant un seuil	Should have
SC8	Historique des transactions	En tant que Banque Centrale, je veux consulter toutes les transactions effectuées	Should have
SC9	Limites quotidiennes	En tant que Banque Centrale, je veux limiter le volume de transferts par banque	Could have
SC10	Événements	En tant que système, je veux émettre des événements pour l'audit et la traçabilité	Must have

Table 3: Sprint Backlog – Smart Contract

3.4 Modèle de Données Blockchain

- Bank pour les informations bancaires (nom, pays, devise, solde, statut d'enregistrement) ;
- PendingTransfer pour les transferts nécessitant approbation ;
- Transaction pour l'historique on-chain des opérations ;
- Des mappings pour accéder rapidement aux banques, aux blacklistes et aux limites quotidiennes.

3.5 Gestion des Permissions

Le contrat implémente le modificateur `onlyOwner()` afin de restreindre les actions critiques à la Banque Centrale : enregistrement des banques, ajout de solde, mise à jour des taux, modification des frais, gestion de la blacklist et approbation des transferts volumineux. Cette approche s'inspire des bonnes pratiques de sécurité recommandées par l'OWASP [12].

3.6 Flux de Transfert dans le Smart Contract

1. Vérification de l'enregistrement de la banque émettrice ;
2. Vérification de l'enregistrement de la banque destinataire ;
3. Vérification de la blacklist pour les deux parties ;
4. Vérification du solde disponible ;
5. Vérification de la limite quotidienne ;
6. Si le montant dépasse `approvalThreshold`, création d'un transfert en attente ;
7. Sinon exécution immédiate via `_executeTransfer()`.

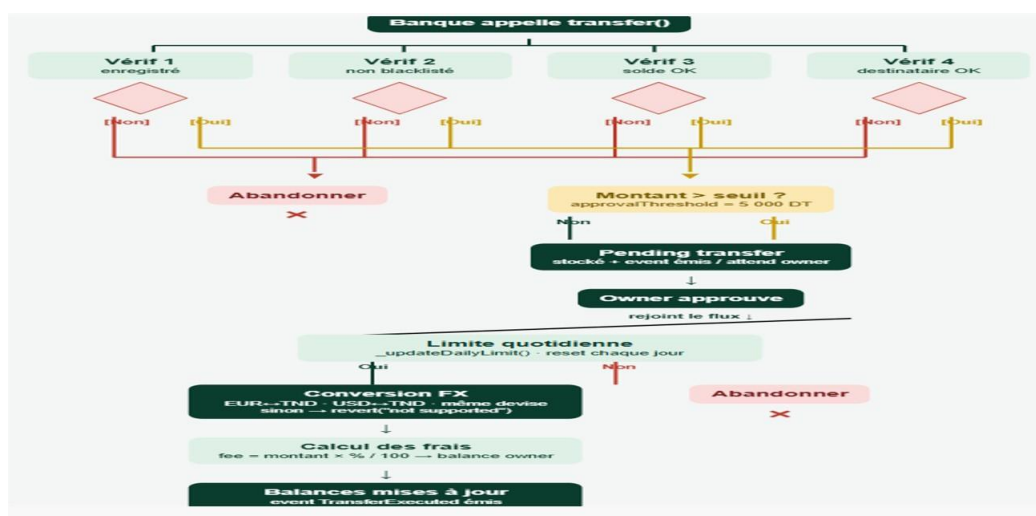


Figure 8: Flux de transfert dans le smart contract

3.7 Traitement des Transferts Volumineux

Les gros transferts sont gérés avec une structure PendingTransfer et un compteur pendingCount. Lorsqu'un transfert est signalé comme « pending », l'événement PendingTransferCreated est émis et le backend enregistre l'opération en base PostgreSQL.

3.8 Exécution Interne et Conversion

- Mise à jour de la limite quotidienne par banque ;
- Déduction du solde émetteur ;
- Conversion de devise lorsque nécessaire ;
- Calcul des frais sur le montant converti ;
- Transfert du montant net vers le destinataire ;
- Crédit du montant des frais vers le propriétaire du contrat ;
- Création et stockage d'une transaction historique.

3.9 Taux de Change et Frais

Les taux de change eurToTndRate et usdToTndRate sont stockés dans le smart contract au format wei. Bien qu'initialisés avec des valeurs par défaut, ils sont mis à jour dynamiquement par le backend via l'API ExchangeRate-API. Le pourcentage des frais de transfert (transferFeePercent) est quant à lui plafonné à 10 % afin de prévenir tout abus et de garantir un cadre sécurisé.

3.10 Blacklist AML

blacklistBank(address) place une banque sur la liste noire, removeFromBlacklist(address) retire une banque de la blacklist, la fonction transfer() bloque toute transaction impliquant une adresse blacklistée, conformément aux recommandations du GAFI [6].

3.11 Événements et Audit

- TransferExecuted pour un transfert finalisé ;
- PendingTransferCreated pour un transfert en attente ;
- PendingTransferApproved pour un transfert approuvé ;
- BalanceAdded pour l'injection de solde.

3.12 Tests et Validation

Pour valider le bon fonctionnement du smart contract, une suite de tests automatisés a été développée avec le framework Hardhat [10]. Ces tests couvrent l'ensemble des fonctionnalités critiques du contrat.


```
PS C:\Users\MSI\Desktop\PP\crossborder-frontend\crossborder-frontend\blockchain2> npx hardhat test
  ✓ ✓ L'owner devrait pouvoir approuver un transfert en attente
  ✓ ✗ Ne devrait pas exécuter deux fois le même transfert en attente
  ✓ ✗ Seul l'owner peut approuver
Historique des Transactions
  ✓ ✓ Devrait stocker les détails de chaque transaction
  ✓ ✓ Devrait incrémenter le compteur de transactions
Calcul et Distribution des Frais
  ✓ ✓ La banque centrale devrait recevoir les frais
  ✓ ✓ Les frais devraient être correctement déduits
Fonctions de Consultation
  ✓ ✓ getBalance() devrait retourner le solde correct
  ✓ ✓ getTransactionCount() devrait retourner le nombre correct
  ✓ ✓ banks() mapping devrait retourner les bonnes infos

43 passing (1s)
```

Figure 9: Résultats des tests Hardhat

Ces 43 tests unitaires valident l'intégralité des fonctionnalités du smart contract CrossBorderPayment : enregistrement des banques, blacklist AML, taux de change, conversions multi-devises et mécanisme multi-signature pour les transferts sensibles. Tous les tests ont été exécutés avec succès en 1 seconde, confirmant la robustesse du contrat et sa conformité aux exigences bancaires.

3.13 Limites et Améliorations Possibles

- Support de nouvelles devises et paires de conversion ;
- Auditabilité multi-signatures pour l'approbation des transferts ;
- Quota dynamique de frais suivant le type de transaction ;
- Intégration d'un mécanisme de suspension d'urgence (circuit breaker).

3.14 Conclusion

Le smart contract constitue le cœur de confiance du projet. Il encadre les règles de transfert, la gestion AML et les conversions de devises, tout en exposant des événements qui permettent au reste du système de construire un historique cohérent. Les chapitres suivants décrivent la réalisation des sprints fonctionnels qui exploitent ce contrat, chacun combinant backend, frontend et intégration blockchain autour d'un périmètre métier cohérent.

4 Sprint 2 — Onboarding et Gestion des Banques

Ce chapitre présente le premier sprint fonctionnel de la plateforme : l'authentification des utilisateurs et la gestion complète du cycle de vie des banques partenaires, depuis leur enregistrement on-chain jusqu'à leur suivi côté interface. Ce sprint livre une tranche verticale complète — backend, base de données, intégration blockchain et écrans Angular associés — plutôt qu'une seule couche technique.

4.1 Sprint Backlog

ID	Fonctionnalité	User Story	Priorité
OB1	Authentification JWT	En tant qu'utilisateur, je veux m'authentifier avec JWT pour accéder à la plateforme	Must have
OB2	Gestion des rôles	En tant qu'utilisateur, je veux avoir un rôle (employé ou Banque Centrale) pour des permissions adaptées	Must have
OB3	Enregistrement des banques	En tant que Banque Centrale, je veux enregistrer une banque via le smart contract	Must have
OB4	Consultation des banques	En tant qu'utilisateur, je veux consulter la liste des banques enregistrées	Must have
OB5	Consultation des soldes	En tant qu'employé de banque, je veux consulter le solde de ma banque	Must have
OB6	Ajout de solde	En tant que Banque Centrale, je veux ajouter du solde à une banque	Should have
OB7	Sécurisation des routes	En tant que système, je veux protéger les routes avec des guards	Must have
OB8	Gestion des erreurs	En tant que système, je veux gérer les erreurs de validation et blockchain	Should have

Table 4: Sprint Backlog — Onboarding et gestion des banques

4.2 Authentification et Gestion des Rôles

L'authentification repose sur JWT. Les mots de passe sont hachés avec bcrypt avant stockage. Deux profils sont gérés : l'employé de banque, rattaché à une banque enregistrée, et la Banque Centrale, disposant de droits d'administration étendus. Le backend protège les endpoints critiques et valide les permissions métier avant tout appel blockchain.

4.2.1 Routes d'Authentification

- /api/auth/login pour la connexion ;
- /api/auth/register pour la création d'un compte employé ;
- /api/auth/forgot-password et /api/auth/reset-password pour la réinitialisation du mot de passe ;
- /api/auth/verify-email pour la vérification d'email ;
- /api/auth/me pour la récupération des informations de l'utilisateur connecté.

4.2.2 Sécurisation de la Navigation

- appAccessGuard pour les utilisateurs authentifiés ;
- centralBankGuard pour la Banque Centrale ;
- transferGuard pour les opérations de transfert.

4.2.3 Écran de Connexion

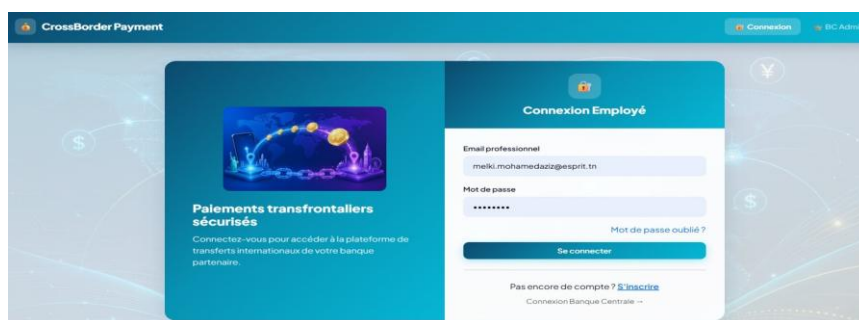


Figure 10: Écran de connexion

L'interface de connexion suit une approche minimaliste et sécurisée éliminant toute complexité superflue. Le design épuré guide l'utilisateur vers l'action principale « Se connecter » tout en offrant un retour immédiat via le lien « Retour aux banques » pour faciliter la navigation. Cette simplicité renforce la confiance des administrateurs en réduisant les risques d'erreur lors de l'authentification.

4.2.4 Authentification en tant qu'Employé de Banque

L'interface de connexion employé est conçue pour offrir une expérience utilisateur fluide et intuitive. Elle intègre deux champs principaux (email et mot de passe) avec une option « Mot de passe oublié ? » pour faciliter la récupération d'accès, renforçant ainsi l'expérience utilisateur en cas de besoin d'assistance. La présence des liens « S'inscrire » et « Connexion Banque Centrale » permet une navigation claire entre les différents rôles du système.

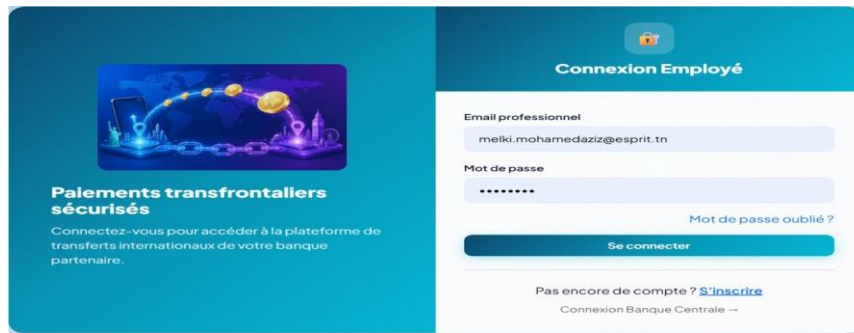


Figure 11: Authentification employé de banque

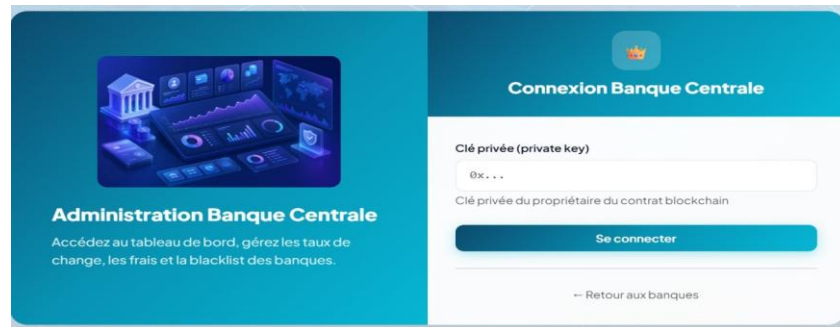


Figure 12: Authentification Banque Centrale

4.2.5 Authentification en tant que Banque Centrale

4.3 Gestion des Banques

- Enregistrement sécurisé des banques sur le smart contract via `registerBank()`;
- Ajout et consultation du solde d'une banque ;
- Consultation de la liste des banques partenaires et de leur statut.

4.3.1 Intégration Blockchain

Le backend connecte un wallet de la Banque Centrale à un nœud Ethereum via WebSocket et utilise Ethers.js pour appeler les fonctions du smart contract liées aux banques (`registerBank()`, `addBalance()`, lecture des soldes on-chain).

4.3.2 Écran de Gestion des Banques

Cette page permet aux administrateurs de consulter la liste des banques enregistrées, leurs soldes et leurs informations clés (pays, devise). Elle offre également la fonctionnalité d'ajout de nouvelles banques, centralisant ainsi la gestion du réseau d'institutions financières participant à la plateforme.

#	NOM	PAYS	DEVISE	BALANCE	LIMITE QUOTIDIENNE	ADRESSE	CRÉÉ LE
1	STB	Tunisia	TND	10001000 TND		0x3C44CDE...	02/06/2026 01:25
2	Deutsche Bank	Germany	EUR	250000293.759829085190350434 TND		0x15d34AA...	04/03/2026 06:35
3	BIAT	Tunisia	TND	1250999500.0 TND		0x79997979...	04/03/2026 06:33
4	JPMorgan Chase	USA	USD	50000335.581844683231076913 TND		0x98798f6...	27/02/2026 22:12
5	BNP Paribas	France	EUR	100001800.0 TND		0xF8B8b9...	26/02/2026 21:28
6	UBCI	Tunisia	TND	159995000.0 TND		0x715d63f3...	26/02/2026 21:27

Figure 13: Écran de gestion des banques

4.4 Modèle de Données

- banks pour les institutions financières : wallet_address, name, country, currency, tx_hash, balance, created_at;
- employees pour l'authentification des utilisateurs.

	wallet_address text	name text	country text	currency text	tx_hash text	created_at timestamp without time zone	balance numeric
1	0x71b653f2384f5fb9895898A86B02Fb2426c5788	UBCI	Tunisia	TND	0x432aae79ac43fca072d2fc15f457c1f46ebd6ccadce819b97112a55db3f6308	2026-02-26 20:27:00.84272	
2	0xfAB80ac9d6880B445fB7357272f2020c5651694a	BNP Paribas	France	EUR	0x2a7900fd64a4ba74f372986d7cc075991eac49e32299aaa20c917ab7427f03	2026-02-26 20:28:07.819156	10000205
3	0x90F79b6EB2c4f870365E785982E1f101E93b906	JPMorgan Chase	USA	USD	0xd33f147e06432b68ce27d684a6a7d408890d1ad1dae044a0cbd0b8b6244f60	2026-02-27 21:12:26.552168	5000034
4	0x70997970C51812dc3A010C7d01b50e0d17dc79...	BIAT	Tunisia	TND	0xded11e0036cfcdc3a1f2d2b690e404d09209c544d8052c53a3d0d9f9601b5f06	2026-03-04 05:33:31.958563	
5	0x15d34AA542670B7D7c367839AA71A00a206A...	Deutsche Bank	Germany	EUR	0x7c6db2e2982be707aa796033d645de6ee087445935b8ca7b33a481cae8dcdb...	2026-03-04 05:35:47.676716	25000225
6	0x3C44Cd0B6a900fa2b585d0299e03d12FA4293BC	STB	Tunisia	TND	0x6dff0575764fd805d511b246294b898f970c060b688b5ecb67ab46f77592dd98	2026-06-01 23:25:30.243143	

Figure 14: Schéma PostgreSQL — Banques

Cette table banks liste les 6 institutions financières enregistrées avec leurs adresses blockchain, leurs soldes et leur historique de transactions. Les banques sont réparties sur plusieurs pays (Tunisie, France, USA, Allemagne) avec des devises variées (TND, EUR, USD) et des soldes. Cette structure permet de gérer l'ensemble des participants au système et d'assurer le suivi des opérations interbancaires [11].

4.5 Robustesse Applicative : Gestion des Erreurs et Stratégie de Tests

La fiabilité de ce sprint repose sur deux piliers complémentaires : une gestion rigoureuse des erreurs à chaque niveau de la chaîne applicative, et une stratégie de tests couvrant à la fois l'API et le smart contract.

Concernant la gestion des erreurs, trois catégories distinctes sont traitées de manière structurée. Les erreurs de validation interceptent les données incohérentes ou incomplètes envoyées par le frontend avant qu'elles n'atteignent la base de données ou la blockchain — par exemple une adresse de wallet mal formée ou un champ obligatoire manquant lors de l'enregistrement d'une banque. Les erreurs PostgreSQL couvrent les cas de contraintes violées, de connexions

perdues ou de conflits d'unicité (comme une tentative d'enregistrement d'une banque déjà présente en base). Enfin, les erreurs propres à la blockchain — reverts du smart contract, délais d'attente dépassés (timeouts) lors d'un appel à un nœud Ethereum, ou échecs de confirmation de transaction — sont capturées séparément, car elles nécessitent souvent une nouvelle tentative ou une notification spécifique à l'utilisateur. Chacune de ces catégories est journalisée côté serveur et renvoyée au frontend sous une forme structurée et homogène (code d'erreur, message explicite, contexte), ce qui permet à l'interface Angular d'afficher un retour compréhensible à l'utilisateur plutôt qu'un message technique brut.

Cette gestion des erreurs a été validée par une campagne de tests couvrant les principaux scénarios du sprint. Les parcours d'inscription et de connexion ont été testés à l'aide de la collection Postman, en simulant à la fois des cas nominaux (identifiants valides) et des cas d'erreur (mot de passe incorrect, compte déjà existant, jeton expiré), afin de vérifier que l'API renvoie les codes de statut et les messages attendus dans chaque situation. En parallèle, des scénarios Hardhat ont permis de valider le comportement de la fonction `registerBank()` côté smart contract, notamment le rejet d'un enregistrement en double ou d'un appel émis par une adresse ne disposant pas des droits nécessaires. Cette double validation, côté API et côté contrat, garantit que les règles métier sont appliquées de façon cohérente sur l'ensemble de la chaîne, du formulaire Angular jusqu'à la blockchain.

4.6 Conclusion

Ce sprint a livré un socle fonctionnel complet : authentification sécurisée par rôle et gestion du cycle de vie des banques partenaires, de l'enregistrement on-chain jusqu'à l'affichage côté interface. Le chapitre suivant s'appuie sur ce socle pour développer le cœur métier de la plateforme : la création et le suivi des transferts.

5 Sprint 3 — Transferts Interbancaires

Ce chapitre présente le deuxième sprint fonctionnel, consacré au cœur métier de la plateforme : la création, l'exécution, l'approbation et le suivi des transferts interbancaires, de la saisie utilisateur jusqu'à l'exécution on-chain.

5.1 Sprint Backlog

ID	Fonctionnalité	User Story	Priorité
TR1	Création de transfert	En tant qu'employé de banque, je veux créer un transfert vers une autre banque	Must have
TR2	Calcul des frais	En tant que système, je veux calculer automatiquement les frais de transfert	Must have
TR3	Conversion de devise	En tant que système, je veux convertir les montants selon les taux en vigueur	Must have
TR4	Transferts en attente	En tant que Banque Centrale, je veux approuver les transferts dépassant le seuil	Should have
TR5	Historisation	En tant que système, je veux historiser toutes les transactions en base PostgreSQL	Must have
TR6	Suivi des transferts	En tant qu'employé de banque, je veux suivre le statut de mes transferts	Should have
TR7	Limites quotidiennes	En tant que Banque Centrale, je veux appliquer des limites quotidiennes par banque	Could have

Table 5: Sprint Backlog — Transferts interbancaires

5.2 Parcours Utilisateur

1. L'employé de banque saisit un transfert via un formulaire guidé ;
2. Le composant Angular appelle le service ApiService;
3. Le backend valide les données, calcule la conversion et les frais, puis appelle le smart contract ;
4. Si le montant dépasse le seuil d'approbation, le transfert bascule en état « pending » et attend la validation de la Banque Centrale ;
5. Sinon le transfert est exécuté immédiatement et historisé ;

6. L'interface est mise à jour avec le statut final.

5.3 Création de Transferts

Les transferts sont créés via des formulaires guidés. Le système vérifie la validité des données, met à jour les taux en temps réel et propose un aperçu du montant converti avant validation. Si le transfert dépasse le seuil d'approbation, il bascule en état « pending ».

Formulaire de transfert

Transfert international : depuis votre banque tunisienne vers une banque étrangère enregistrée.

Banque émettrice
UBCI(0x71bE63f3...)
Votre banque (fixée selon votre compte employé)

Clé privée de la banque émettrice
0x...
Doit correspondre à UBCI(0x71bE63f3...)

Banque destinataire (étranger)
-- Sélectionner une banque étrangère --

Montant (TND)
1000
Montant > 5000 TND : approbation requise par la Banque Centrale

Transférer **Annuler**

Figure 15: Formulaire de création de transfert

Cette interface présente le formulaire guidé pour initier un nouveau transfert interbancaire. L'utilisateur sélectionne la banque émettrice, la banque destinataire, saisit le montant et la devise. Le système affiche un aperçu de la conversion et des frais avant la soumission, améliorant la transparence et limitant les erreurs.

5.4 Validation des Transferts en Attente

La Banque Centrale dispose d'un écran dédié pour consulter et approuver les transferts dépassant le seuil, garantissant un contrôle humain sur les opérations sensibles.

Transferts en attente
Transferts en attente d'approbation par la Banque Centrale

2 en attente
Validation requise pour les montants supérieurs à 5000 TND

ID	DE	VERS	MONTANT	STATUT	DATE	ACTION
#0	0x71b63...	0x15d34A...	188888 TND	pending	Mar 4, 2026, 7:30:26AM	Approuver
#3	0x71b63...	0xfAB00a...	144444 TND	approved	Feb 26, 2026, 10:18:53PM	✓ Approuvé

Figure 16: Validation des transferts en attente

Cette vue est dédiée aux administrateurs de la Banque Centrale pour gérer les transferts dépassant le seuil d'approbation automatique (5000 DT). Elle liste les opérations en attente avec leurs détails (banques concernées, montant, date) et permet à l'administrateur d'approuver ou de rejeter chaque transfert après un examen minutieux.

5.5 Exécution et Historisation

- Mise à jour de la limite quotidienne par banque ;
- Déduction du solde émetteur et conversion de devise lorsque nécessaire ;
- Calcul des frais et transfert du montant net vers le destinataire ;
- Création et stockage d'une transaction historique en base PostgreSQL.

5.6 Traitement des Événements Blockchain

- PendingTransferCreated : insertion dans pending_transfers;
- PendingTransferApproved : mise à jour du statut et insertion dans transactions;
- TransferExecuted : historique des transactions.

5.7 Modèle de Données

- transactions : from_bank, to_bank, original_amount, converted_amount, fee, tx_hash, block_number, created_at;
- pending_transfers : transfer_id, from_bank, to_bank, amount, status, approved_at, created_at.

id [PK] integer	from_bank text	to_bank text	original_amount text	converted_amount text	fee text
1	0x90F79b6EB2c4f870365E785982E1f101E93b906	0x9965507D1a55bcC2695C58ba16FB37d819B0A...	5000.0	15345.0	155.0
2	0x90F79b6EB2c4f870365E785982E1f101E93b906	0x9965507D1a55bcC2695C58ba16FB37d819B0A...	5000.0	15345.0	155.0
3	0x70997970C51812dc3A010C7d01b50e0d17dc7...	0xf39fd6e51aad88f6F4ce6aB8827279cFFb92266	2000.0	6534.0	66.0
4	0x71bE63f3384f5fb98995898A86B02Fb2426c5788	0x3C44CdDdB6a900fa2b585dd299e03d12FA4293...	1000.0	300.0	3.0303030303030303
5	0x71bE63f3384f5fb98995898A86B02Fb2426c5788	0x3C44CdDdB6a900fa2b585dd299e03d12FA4293...	6000.0	1800.0	18.1818181818181818
6	0xd2FD4581271e230360230F9337D5c0430Bf4...	0xFABBOac9d68B0B445fB7357272F202C5651694a	1000.0	300.0	3.0303030303030303
7	0xd2FD4581271e230360230F9337D5c0430Bf4...	0xFABBOac9d68B0B445fB7357272F202C5651694a	1000.0	1000.0	10
8	0xd2FD4581271e230360230F9337D5c0430Bf4...	0xFABBOac9d68B0B445fB7357272F202C5651694a	1000.0	300.0	3.0303030303030303
9	0xd2FD4581271e230360230F9337D5c0430Bf4...	0xFABBOac9d68B0B445fB7357272F202C5651694a	1000.0	1000.0	10
10	0xd2FD4581271e230360230F9337D5c0430Bf4...	0xFABBOac9d68B0B445fB7357272F202C5651694a	5500.0	5500.0	0
11	0xd2FD4581271e230360230F9337D5c0430Bf4...	0xFABBOac9d68B0B445fB7357272F202C5651694a	5500.0	1650.0	16.6666666666666666
12	0xa0Ee7A142d267C1f36714E4a8F75612F20a797...	0x8626f6940E2eb28930eFb4Cef49B2d1F2C9C11...	1000.0	300.0	3.0303030303030303
13	0xa0Ee7A142d267C1f36714E4a8F75612F20a797...	0x8626f6940E2eb28930eFb4Cef49B2d1F2C9C11...	1000.0	1000.0	10
14	0xa0Ee7A142d267C1f36714E4a8F75612F20a797...	0x8626f6940E2eb28930eFb4Cef49B2d1F2C9C11...	5500.0	5500.0	0
15	0xd2FD4581271e230360230F9337D5c0430Bf4...	0x8626f6940E2eb28930eFb4Cef49B2d1F2C9C11...	1000.0	1000.0	10
16	0x9965507D1a55bcC2695C58ba16FB37d819B0A...	0x8626f6940E2eb28930eFb4Cef49B2d1F2C9C11...	1100.0	330.0	3.3333333333333333
17	0x9965507D1a55bcC2695C58ba16FB37d819B0A...	0x8626f6940E2eb28930eFb4Cef49B2d1F2C9C11...	1100.0	333.33333333333337	3.3333333333333334
18	0x71bE63f3384f5fb98995898A86B02Fb2426c5788	0xFABBOac9d68B0B445fB7357272F202C5651694a	1000.0	300.0	3.0303030303030303
19	0x71bE63f3384f5fb98995898A86B02Fb2426c5788	0xFABBOac9d68B0B445fB7357272F202C5651694a	1000.0	303.03030303030306	3.0303030303030307

Figure 17: Schéma PostgreSQL — Transferts

Cette capture présente des transactions enregistrées dans la base de données PostgreSQL du système CrossBorder Payment. Les montants originaux varient avec des conversions et des frais appliqués automatiquement, comme en témoignent les valeurs de converted_amount et fee. Les adresses wallet des banques émettrices et destinataires sont tracées, reflétant la diversité des transferts interbancaires effectués via la plateforme.

5.7.1 Exemples de Vues PostgreSQL

- bank_transaction_volume : volume envoyé par banque ;
- latest_transactions_detailed : historique enrichi avec noms et pays.

5.8 Fiabilisation du Parcours de Transfert : Gestion des Erreurs et Validation par les Tests

Le parcours de transfert étant la fonctionnalité la plus sensible de la plateforme, une attention particulière a été portée à sa robustesse, à travers une gestion différenciée des erreurs et une couverture de tests répartie sur les trois couches du système.

Le backend distingue trois familles d'erreurs susceptibles de survenir lors d'un transfert. Les erreurs de validation interceptent les incohérences détectées avant tout appel externe : montant négatif ou nul, devise non supportée, banque destinataire inexistante ou identique à la banque émettrice. Les erreurs PostgreSQL couvrent les échecs liés à la persistance des données — perte de connexion à la base au moment de l'historisation, ou violation de contrainte lors de l'insertion d'un transfert en attente. Les erreurs blockchain, enfin, regroupent les cas de revert du smart contract (par exemple lorsque le solde de la banque émettrice est insuffisant ou que l'une des deux banques est blacklistée) ainsi que les situations de timeout lors de l'attente de confirmation d'une transaction sur le réseau Ethereum. Chacune de ces erreurs est journalisée avec son contexte d'origine puis renvoyée au frontend sous un format structuré, ce qui permet à l'employé de banque de comprendre immédiatement la cause d'un échec plutôt que de faire face à un blocage silencieux.

Cette gestion des erreurs a été éprouvée par des scénarios de test couvrant l'ensemble du parcours de transfert sur les trois couches de l'application : smart contract, API et interface. Trois cas principaux ont été systématiquement vérifiés : la création d'un transfert simple, sous le seuil d'approbation, qui doit s'exécuter immédiatement et apparaître dans l'historique ; la création d'un transfert dépassant le seuil d'approbation, qui doit basculer en état « pending » sans être exécuté ; et la validation de ce transfert en attente par la Banque Centrale, qui doit déclencher son exécution effective et la mise à jour cohérente du statut sur les trois couches. Cette approche multi-niveaux garantit que le comportement observé côté interface reflète fidèlement l'état réel du smart contract et de la base de données.

5.9 Conclusion

Ce sprint a livré le cœur métier de la plateforme : un parcours de transfert complet, sécurisé et traçable, de la saisie à l'exécution on-chain. Le chapitre suivant présente les fonctionnalités de conformité et de pilotage : taux de change, blacklist AML et tableau de bord.

6 Sprint 4 — Conformité, Taux de Change et Pilotage

Ce chapitre présente le troisième sprint fonctionnel, consacré aux fonctionnalités de conformité réglementaire (blacklist AML), à la gestion dynamique des taux de change et de frais, ainsi qu'au tableau de bord de pilotage destiné à la Banque Centrale.

6.1 Sprint Backlog

ID	Fonctionnalité	User Story	Priorité
CP1	Blacklist AML	En tant que Banque Centrale, je veux ajouter/retirer des banques de la blacklist	Must have
CP2	Vérification blacklist	En tant que système, je veux bloquer les transferts impliquant une banque black-listée	Must have
CP3	Taux de change	En tant que Banque Centrale, je veux mettre à jour les taux EUR/TND et USD/TND	Must have
CP4	Synchronisation automatique	En tant que système, je veux synchroniser les taux via une API externe	Should have
CP5	Frais de transfert	En tant que Banque Centrale, je veux configurer les frais de transfert	Must have
CP6	Tableau de bord KPI	En tant que Banque Centrale, je veux consulter les indicateurs clés	Should have
CP7	Notifications	En tant que système, je veux envoyer des alertes par email	Could have
CP8	Historique des taux	En tant que Banque Centrale, je veux consulter l'historique des taux	Could have

Table 6: Sprint Backlog — Conformité, taux de change et pilotage

6.2 Blacklist AML

- `blacklistBank(address)` place une banque sur la liste noire ;
- `removeFromBlacklist(address)` retire une banque de la blacklist ;
- La fonction `transfer()` bloque toute transaction impliquant une adresse blacklistée.

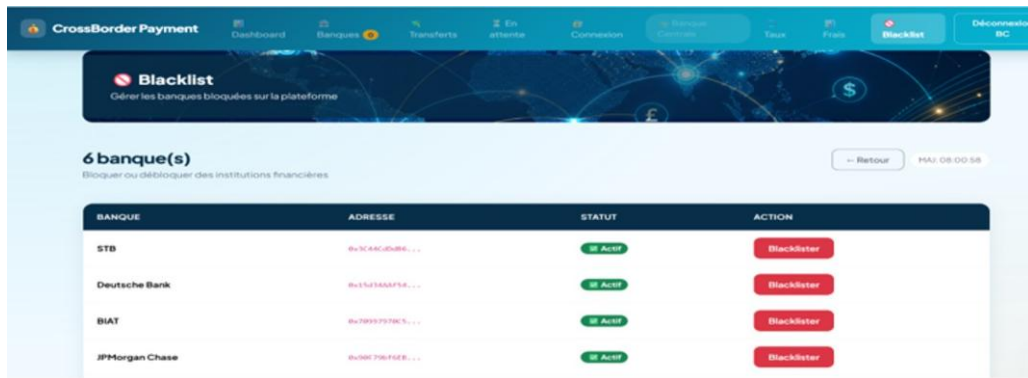


Figure 18: Administration de la blacklist AML

6.2.1 Écran d'Administration de la Blacklist

Cette capture présente la page d'administration de la blacklist AML du système CrossBorder Payment. Elle liste 6 banques avec leur statut « Actif », permettant à la Banque Centrale de bloquer ou débloquer des institutions financières en un clic via le bouton « Blacklister ». Cette interface intuitive garantit une gestion rapide et efficace de la conformité réglementaire, essentielle pour la sécurité des transferts interbancaires [6].

6.3 Taux de Change et Frais

Le contrat expose les taux `eurToTndRate` et `usdToTndRate` en format `wei`. Ces taux sont initialisés dans le contrat, puis le backend met à jour leurs valeurs dynamiquement via `ExchangeRate-API`. Les frais de transfert (`transferFeePercent`) sont plafonnés à 10% pour éviter tout abus.

6.3.1 Automatisation

Un cron job quotidien exécute la synchronisation des taux via le service `exchange-rate.service.js` qui récupère et met à jour les valeurs depuis `ExchangeRate-API` toutes les 24 heures.

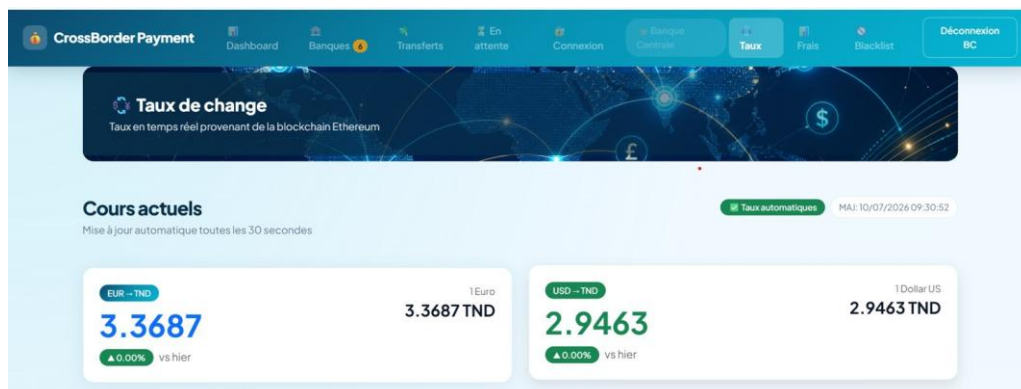


Figure 19: Gestion des taux de change

Cette capture affiche la page de gestion des taux de change en temps réel de la plateforme Cross-Border Payment. Les taux EUR/TND (3.3687) et USD/TND (2.9463) sont automatiquement mis à jour toutes les 30 secondes depuis la blockchain Ethereum, garantissant une précision

constante. L'interface intuitive présente les variations par rapport à la veille (0.00%), offrant aux banques une visibilité claire et immédiate sur les conditions du marché.

6.4 Tableau de Bord et Pilotage

Le tableau de bord offre une vision synthétique des indicateurs clés de performance, de la répartition des transactions par devise et par pays, ainsi que de l'activité récente du système.

6.4.1 Écran du Tableau de Bord

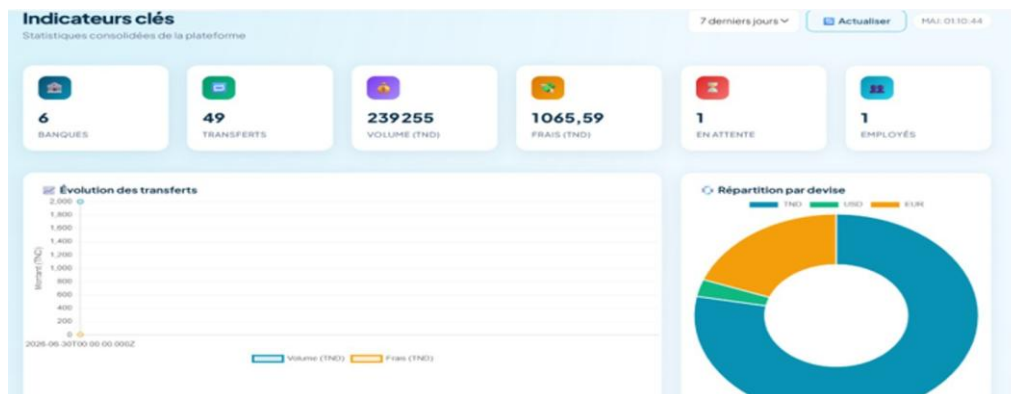


Figure 20: Tableau de bord de pilotage

Cette capture montre le tableau de bord dynamique de la Banque Centrale. Il synthétise les indicateurs de performance essentiels tels que le nombre de transferts, les volumes échangés, l'évolution des transactions, et la répartition par devise/pays, offrant ainsi une vision stratégique en temps réel de l'activité de la plateforme.

6.5 Modèle de Données

- blacklist_history pour l'audit des actions de blacklist ;
- rate_history pour l'historique des taux Forex.

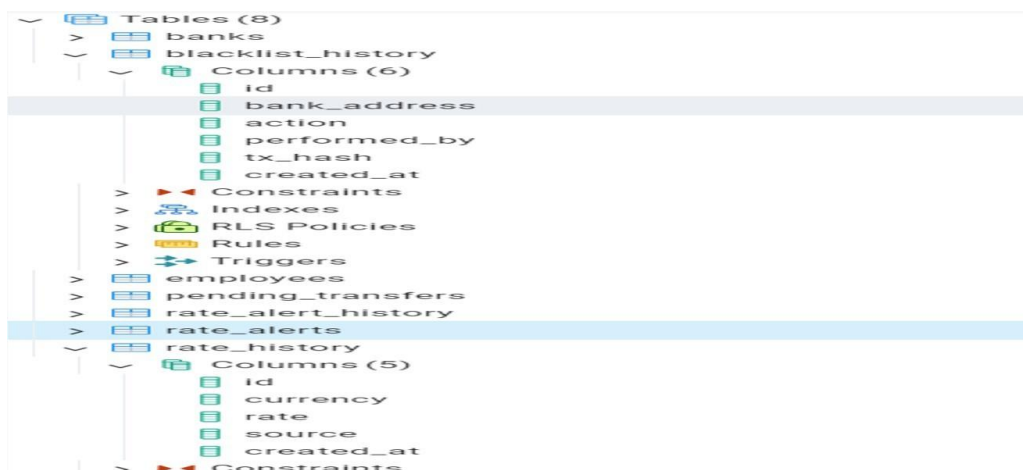


Figure 21: Schéma PostgreSQL — Taux et blacklist

6.5.1 Exemples de Vues PostgreSQL

currency_stats centralise les statistiques agrégées par devise, utilisées pour alimenter les graphiques du tableau de bord.

6.6 Notifications et Alertes

Le backend utilise nodemailer pour envoyer des emails de notification concernant les transferts en attente, les changements de taux importants et les actions administratives.

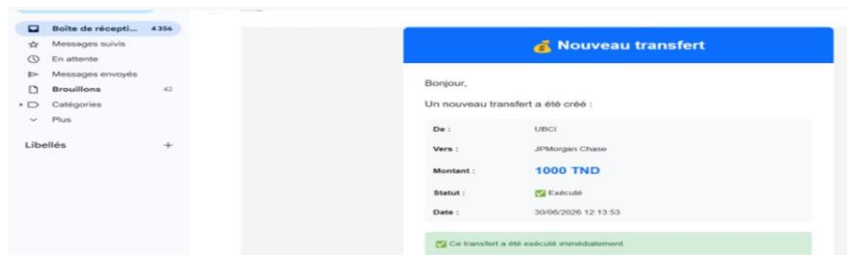


Figure 22: Notification par email

6.7 Tests et Validation

Des scénarios de test valident le blocage effectif d'une adresse blacklistée lors d'un transfert, la mise à jour correcte des taux et des frais, ainsi que la cohérence des indicateurs affichés sur le tableau de bord.

6.8 Conclusion

Ce sprint a doté la plateforme de ses outils de conformité et de supervision : blacklist AML opérationnelle, gestion dynamique des taux et tableau de bord de pilotage pour la Banque Centrale. Le chapitre suivant présente les fonctionnalités de reporting et l'industrialisation du déploiement.

7 Sprint 5 — Reporting et Industrialisation

Ce chapitre présente le dernier sprint fonctionnel, consacré à l'export de rapports, à la containerisation de la plateforme et à la préparation d'un pipeline d'intégration continue.

7.1 Sprint Backlog

ID	Fonctionnalité	User Story	Priorité
RI1	Export PDF	En tant que Banque Centrale, je veux exporter les rapports en PDF	Could have
RI2	Containerisation Docker	En tant que développeur, je veux containeriser l'application pour faciliter le déploiement	Should have
RI3	CI/CD	En tant que développeur, je veux préparer un pipeline d'intégration continue	Could have
RI4	Documentation Swagger	En tant que développeur, je veux documenter les API avec Swagger	Should have
RI5	Tests d'intégration	En tant que système, je veux automatiser les tests d'intégration	Could have

Table 7: Sprint Backlog — Reporting et industrialisation

7.2 Export PDF

Le service PdfService, basé sur jsPDF et jspdf-autotable, génère des rapports PDF pour les transferts et les indicateurs du tableau de bord.

Généré le mardi 7 juillet 2026

#	Nom	Pays	Devise	Balance
1	STB	Tunisia	TND	10001000.00
2	Deutsche Bank	Germany	EUR	250000293.76
3	BIAT	Tunisia	TND	1250999500.00
4	JPMorgan Chase	USA	USD	50000335.58
5	BNP Paribas	France	EUR	100001800.00
6	UBCI	Tunisia	TND	159995000.00

Figure 23: Export de rapports PDF

7.3 Architecture Backend Consolidée

Le client Angular envoie ses requêtes à une API Gateway (Express), qui vérifie l'identité de l'utilisateur via un middleware JWT avant de router l'appel vers le module métier concerné

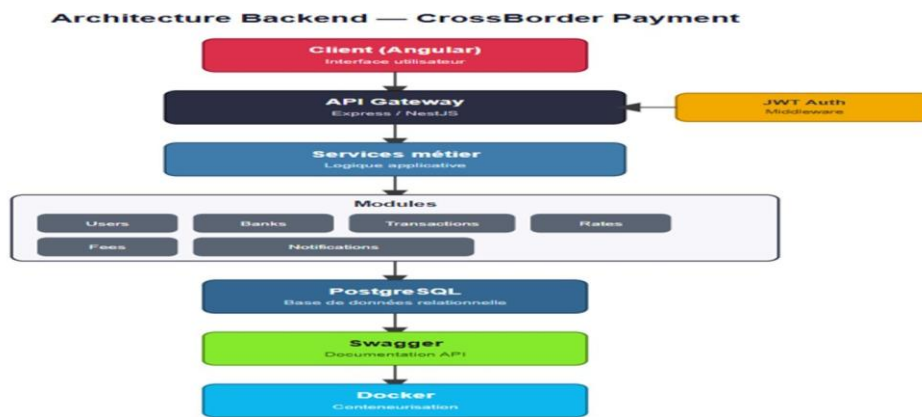


Figure 24: Architecture backend consolidée

(banques, transferts, taux, frais, notifications). Chaque service exécute sa logique et interroge la base PostgreSQL pour lire ou écrire les données [11]. L'ensemble est documenté via Swagger et déployé de façon reproductible grâce à des conteneurs Docker.

7.4 Base de Données PostgreSQL

- banks pour les institutions financières ;
- transactions pour l'historique des transferts enregistrés ;
- pending_transfers pour les transferts en attente d'approbation ;
- blacklist_history pour l'audit des actions de blacklist ;
- employees pour l'authentification des utilisateurs ;
- rate_history pour l'historique des taux Forex ;
- rate_alerts et rate_alert_history pour la gestion des alertes de taux.

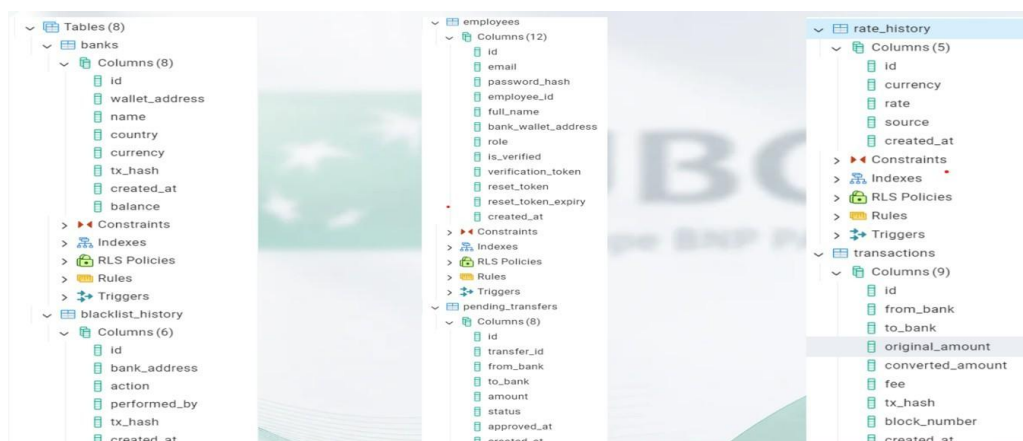


Figure 25: Schéma complet de la base de données PostgreSQL

Cette capture présente le schéma de la base de données PostgreSQL du système CrossBorder Payment, structuré autour de 8 tables interconnectées. Les tables principales incluent banks pour les institutions financières (8 colonnes), employees pour les utilisateurs (12 colonnes)

et transactions pour l'historique des transferts (9 colonnes). Des tables auxiliaires comme `blacklist_history`, `pending_transfers` et `rate_history` assurent respectivement la traçabilité AML, la gestion des transferts en attente et le suivi des taux de change.

7.5 Dockerisation

La plateforme est containerisée à l'aide de Docker, avec des images dédiées pour le frontend, le backend et le nœud blockchain, facilitant le déploiement et la reproductibilité de l'environnement.

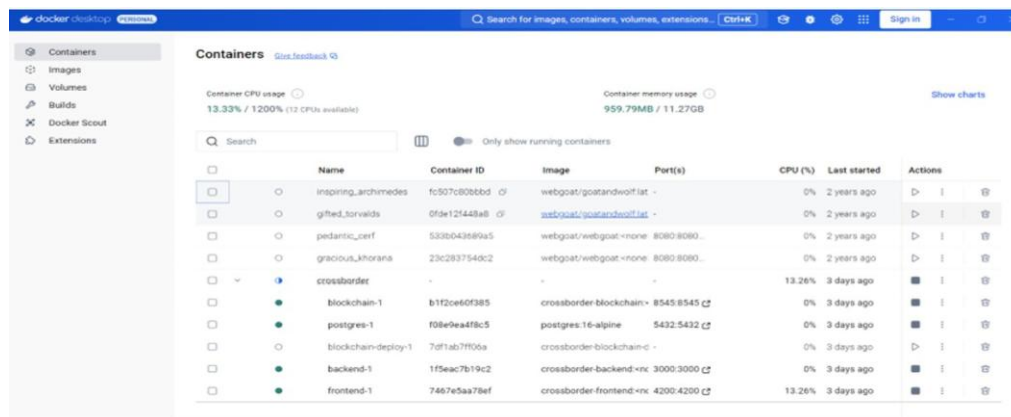


Figure 26: Conteneurs Docker

7.6 Industrialisation du Déploiement : Containerisation, Intégration Continue et Gestion des Risques

Au-delà du reporting, ce dernier sprint a pour vocation de professionnaliser la mise en production de la plateforme. Trois volets complémentaires y contribuent : la containerisation avec Docker, la préparation d'une chaîne d'intégration continue, et une stratégie de tests et de gestion des risques transverse aux trois couches applicatives.

La plateforme a été containerisée à l'aide de Docker, avec des images distinctes pour le frontend Angular, le backend Node.js et le nœud blockchain local. Avant la mise en place de cette containerisation, le lancement de l'application nécessitait l'ouverture de plusieurs terminaux distincts — un pour le frontend, un pour le backend et un pour le réseau Hardhat — ce qui compliquait le déploiement et rendait l'environnement difficilement reproductible d'un poste à l'autre. L'usage de conteneurs dédiés, orchestrés via `docker-compose`, a permis de résoudre ce problème en encapsulant chaque composant avec ses dépendances propres, et de démarrer l'ensemble de la plateforme en une seule commande.

Dans le prolongement de cette containerisation, la mise en place d'un pipeline d'intégration continue (CI/CD) a été envisagée pour automatiser la chaîne de livraison : compilation du frontend Angular, exécution de la suite de tests, construction des images Docker et validation des fichiers de configuration avant tout déploiement. Cette automatisation, bien que préparée conceptuellement durant ce sprint, constitue un axe de poursuite naturel pour fiabiliser les futures évolutions de la plateforme, comme évoqué au chapitre 8.

Sur le plan des tests, la stratégie adoptée couvre les trois couches de l'application — smart contracts, backend et frontend — avec des scénarios de validation définis pour garantir la qualité de service. Une attention particulière a été portée à la validité des rapports PDF générés par le service PdfService, en vérifiant que les données exportées correspondent bien à l'état courant de la base PostgreSQL au moment de la génération. Cette approche de gestion des risques par les tests, combinée à la reproductibilité apportée par Docker, constitue une base solide pour l'industrialisation ultérieure du projet.

7.7 Difficultés Rencontrées

- L'intégration entre backend et smart contract via WebSocket ;
- La gestion des conversions multi-devises ;
- La sécurisation des accès JWT ;
- La synchronisation des données entre frontend et backend ;
- La configuration de la containerisation Docker pour les trois composants ;
- La différenciation des droits entre employés de banque et Banque Centrale ;
- La récupération des données enregistrées sur le réseau blockchain ;
- Le lancement de l'application via plusieurs terminaux pour le frontend, le backend et le réseau blockchain (problème résolu par l'implémentation des conteneurs avec Docker).

7.8 Conclusion

Ce dernier sprint a permis de professionnaliser la plateforme à travers l'export de rapports, la containerisation et la préparation d'une chaîne d'intégration continue, consolidant la solution en vue de son évaluation finale.

8 Évaluation et Perspectives

8.1 Analyse Critique

Ce stage m'a permis de renforcer mes compétences en développement full-stack et blockchain, en couvrant l'ensemble de la chaîne technique : smart contract Solidity [8], backend Node.js [9], base de données PostgreSQL et interface Angular. J'ai acquis une vision concrète des enjeux de sécurité, de conformité et de traçabilité dans le secteur bancaire [7], en mettant en œuvre des mécanismes comme l'authentification JWT, la gestion des rôles et la blacklist AML. Cette expérience a confirmé mon intérêt pour la fintech et les architectures décentralisées, tout en développant mes capacités d'analyse, d'organisation et de collaboration au sein d'une grande institution financière, dans le cadre d'une méthode agile Scrum [3].

Limites et axes d'amélioration : Plusieurs pistes d'évolution ont été identifiées. Sur le plan technique, l'optimisation des performances du tableau de bord (temps de chargement des KPIs) pourrait être améliorée par l'introduction d'un cache Redis. Les tests automatisés, actuellement concentrés sur le smart contract (43 tests unitaires), mériteraient d'être étendus au backend et au frontend. Enfin, l'ajout de nouvelles devises (GBP, JPY, CHF) nécessiterait une évolution du smart contract et de la logique de conversion associée.

Comparaison planning prévisionnel vs planning réalisé : Le planning initial prévoyait un découpage en six sprints. En réalité, le projet s'est articulé autour de six sprints, le dernier étant dédié à la containerisation Docker, à l'export PDF et à la rédaction du rapport. Ce décalage s'explique par la complexité de l'intégration entre le backend et le smart contract via WebSocket, ainsi que par la mise en place de l'authentification JWT, qui ont nécessité des itérations supplémentaires. Le diagramme de Gantt illustre cette évolution et permet de visualiser les ajustements apportés en cours de projet.

8.2 Bilan Personnel

Ce stage de projet de fin d'études a constitué une expérience profondément enrichissante, tant sur le plan technique que sur le plan humain et organisationnel.

Développement des compétences techniques : J'ai renforcé mes compétences en développement full-stack et blockchain en couvrant l'ensemble de la chaîne technique : conception et implémentation d'un smart contract Solidity robuste (350 lignes, 43 tests unitaires), développement d'un backend Node.js/Express avec 23 endpoints REST, modélisation d'une base de données PostgreSQL (8 tables interconnectées) et réalisation d'une interface Angular 21 moderne et ergonomique. La mise en œuvre de la containerisation Docker m'a également permis d'aborder les problématiques de déploiement et d'industrialisation.

Compréhension des enjeux métier : J'ai acquis une vision concrète des exigences du secteur bancaire en matière de sécurité, de conformité et de traçabilité. La mise en œuvre de mécanismes tels que l'authentification JWT, la gestion des rôles (employé de banque vs Banque Centrale), la blacklist AML et la gestion des transferts sensibles (>5000 TND) m'a permis de comprendre les contraintes réglementaires et opérationnelles qui pèsent sur les institutions financières.

Méthodologie et organisation : Ce stage a confirmé mon intérêt pour les méthodologies agiles, en particulier Scrum, que j'ai mise en œuvre de manière autonome tout au long du projet. La planification en sprints, les revues régulières avec mes encadrantes et la gestion du product backlog m'ont apporté une rigueur et une vision structurée du développement.

Projet professionnel : Cette expérience a conforté mon orientation vers les métiers de l'ingénierie logicielle dans le secteur de la fintech et des architectures décentralisées. La découverte des technologies blockchain et smart contracts a éveillé une curiosité que je souhaite approfondir, que ce soit en poursuivant des projets personnels ou en intégrant une équipe dédiée à ces sujets.

Savoir-être : Enfin, cette expérience m'a permis de développer mes capacités d'analyse, d'organisation et de collaboration, grâce aux échanges réguliers avec mes encadrantes (Madame Malek Hedhli et Madame Farah Khelif), à la participation à des revues de projet et à la gestion autonome de mon planning dans le respect des délais.

8.3 Contributions du Projet

Le projet CrossBorder Payment apporte plusieurs contributions concrètes au domaine des transferts interbancaires :

1. Une architecture modulaire et réutilisable : La séparation claire entre le frontend, le backend et la blockchain permet une évolution indépendante de chaque composant et facilite l'ajout de nouvelles fonctionnalités.
2. Un smart contract sécurisé : Le contrat CrossBorderPayment.sol intègre des mécanismes de sécurité avancés (multi-signature, blacklist AML, limites quotidiennes) et a fait l'objet de 43 tests unitaires, garantissant sa robustesse.
3. Un backend performant : L'API REST (23 endpoints) assure la centralisation de la logique métier, la persistance des données et l'orchestration des appels à la blockchain.
4. Une interface utilisateur professionnelle : Le frontend Angular offre une expérience fluide et adaptée aux différents profils (employés de banque, Banque Centrale).
5. Une solution industrialisable : La containerisation Docker et la préparation d'un pipeline CI/CD facilitent le déploiement et la reproductibilité de l'environnement.
6. Une démonstration de faisabilité : Ce projet prouve qu'une architecture hybride combinant blockchain et services traditionnels est pertinente et opérationnelle pour répondre aux enjeux des paiements interbancaires internationaux.

8.4 Perspectives d'Évolution

Plusieurs axes d'amélioration et d'enrichissement sont envisageables pour la suite du projet :

À court terme (3 mois) :

- Prise en charge de nouvelles devises : Ajout des paires GBP/TND, JPY/TND, CHF/TND.
- Authentification multi-facteur (2FA) : Renforcement de la sécurité des comptes employés.

- Optimisation des performances : Introduction d'un cache Redis pour le tableau de bord.
- Extension des tests automatisés : Couverture du backend (Jest) et du frontend (Cypress).

À moyen terme (6 mois) :

- Module de reporting avancé : Création de rapports personnalisables et export Excel.
- Pipeline CI/CD complet : Automatisation des builds, des tests et des déploiements.
- Migration vers un réseau testnet : Déploiement sur Sepolia pour des tests en conditions réelles.
- Interface mobile responsive : Adaptation complète aux écrans mobiles.

À long terme (1 an) :

- Déploiement sur mainnet Ethereum : Mise en production sur le réseau public.
- Intégration avec des systèmes bancaires existants : Connexion avec les API des banques partenaires.
- Ajout d'un module d'analyse prédictive : Utilisation de machine learning pour détecter des tendances et anomalies.

Technologies Utilisées

Pour éviter toute redondance, l'ensemble des technologies mobilisées par le projet CrossBorder Payment est récapitulé ici en un seul endroit, plutôt que d'être répété au fil des chapitres consacrés aux différents sprints.

Frontend

- Angular 21 en mode standalone [9], pour la structure de l'application web ;
- Bootstrap 5 [13], pour la mise en forme et la responsivité de l'interface ;
- ng2-charts, pour les graphiques du tableau de bord ;
- jsPDF et jspdf-autotable, pour la génération des rapports exportés au format PDF.

Backend

- Node.js et Express [9], pour l'API REST et l'orchestration de la logique métier ;
- PostgreSQL [11], pour la persistance relationnelle des données ;
- JSON Web Token (JWT) et bcrypt, pour l'authentification et la sécurisation des mots de passe ;
- Ethers.js, pour l'intégration entre le backend et le smart contract ;
- nodemailer, pour l'envoi des notifications par email ;
- Swagger, pour la documentation des endpoints de l'API.

Blockchain

- Solidity, pour l'écriture du smart contract CrossBorderPayment.sol ;
- Hardhat [10], pour le développement, le déploiement local et les tests automatisés du contrat ;
- Réseau Ethereum local (développement) et réseau Sepolia (tests envisagés).

Infrastructure et Outillage

- Docker et docker-compose, pour la containerisation et le déploiement reproductible des trois composants ;
- Postman, pour les tests manuels et automatisés de l'API REST ;
- Git, pour le contrôle de version du code source.

Conclusion Générale

En conclusion, le projet CrossBorder Payment a abouti à la réalisation d'une plateforme de paiement transfrontalier complète et fonctionnelle, s'appuyant sur une architecture à trois couches — frontend, backend et blockchain — conçue pour garantir sécurité, transparence et efficacité. L'adoption de la méthodologie agile Scrum, avec un découpage en sprints fonctionnels, a permis de structurer le développement de manière itérative et de livrer des incréments de valeur de façon continue, tout en maintenant une cohérence globale entre les différentes couches techniques. La solution répond efficacement à la problématique de départ en automatisant les transferts interbancaires, en assurant leur traçabilité complète et en garantissant la conformité réglementaire, notamment en matière de lutte contre le blanchiment d'argent. L'architecture hybride mise en œuvre, combinant les avantages des services traditionnels [4] et de la blockchain [1], s'avère particulièrement adaptée aux enjeux du secteur bancaire, offrant à la fois robustesse, fiabilité et innovation. Au-delà des résultats obtenus, ce projet constitue une fondation solide pour des évolutions futures, telles que l'ajout de nouvelles devises, le développement d'outils de supervision avancés et l'extension vers des paiements internationaux multi-devises [5]. Cette expérience a également démontré la faisabilité et la pertinence d'une approche hybride dans le domaine des paiements transfrontaliers.

Bibliographie

1. A. Antonopoulos, G. Wood, Mastering Ethereum : Building Smart Contracts and DApps, O'Reilly Media, 2018.
2. S. Nakamoto, Bitcoin : A Peer-to-Peer Electronic Cash System, 2008.
3. K. Schwaber, J. Sutherland, The Scrum Guide, Scrum.org, 2020.
4. R. C. Martin, Clean Architecture : A Craftsman's Guide to Software Structure and Design, Prentice Hall, 2017.
5. Banque des Règlements Internationaux (BRI), Cross-border payments – a vision for the future, Comité sur les paiements et les infrastructures de marché, 2020.
6. GAFI (Groupe d'Action Financière), Recommandations sur la lutte contre le blanchiment de capitaux, 2023.
7. Banque Centrale de Tunisie, Circulaire relative à la sécurité des systèmes de paiement, dernière consultation 2026.
8. V. Buterin, Ethereum Whitepaper, 2013.
9. Solidity Documentation, <https://docs.soliditylang.org/>, consulté en 2026.
10. Angular Documentation, <https://angular.io/>, consulté en 2026.
11. Node.js Documentation, <https://nodejs.org/>, consulté en 2026.
12. PostgreSQL Documentation, <https://www.postgresql.org/>, consulté en 2026.
13. Hardhat Documentation, <https://hardhat.org/>, consulté en 2026.
14. Bootstrap Documentation, <https://getbootstrap.com/>, consulté en 2026.
15. OWASP Foundation, OWASP Smart Contract Top 10, <https://owasp.org/>, consulté en 2026.

Glossaire des Abréviations

Conformité et Pilotage Métier

Sigle	Définition
AML	Anti-Money Laundering — lutte contre le blanchiment d'argent ; désigne dans le rapport la blacklist et les contrôles associés imposés aux transferts interbancaires.
KPI	Key Performance Indicator — indicateur clé de performance, affiché sur le tableau de bord de pilotage de la Banque Centrale (volume, frais, transferts en attente, etc.).
MoSCoW	Must have, Should have, Could have, Won't have — méthode de priorisation des exigences utilisée pour hiérarchiser le product backlog.

Architecture et Développement Logiciel

Sigle	Définition
API	Application Programming Interface — interface de programmation exposée par le backend pour permettre au frontend et à d'autres services d'interagir avec le système.
CI/CD	Continuous Integration / Continuous Deployment — intégration et déploiement continus ; pipeline visant à automatiser la compilation, les tests et la construction des images Docker.
CRUD	Create, Read, Update, Delete — ensemble des opérations de base de manipulation des données dans une architecture REST.
EVM	Ethereum Virtual Machine — machine virtuelle qui exécute le bytecode des smart contracts déployés sur le réseau Ethereum.
JWT	JSON Web Token — format de jeton utilisé pour l'authentification et la gestion des sessions des employés de banque et de la Banque Centrale.
REST	Representational State Transfer — style d'architecture utilisé pour la conception des routes API du backend Node.js/Express.
SGBD	Système de Gestion de Base de Données — ici PostgreSQL, utilisé pour la persistance des données métier (banques, transactions, taux, blacklist).

UML	Unified Modeling Language — langage de modélisation utilisé pour les diagrammes de cas d'utilisation, de classes et de séquence du chapitre 2.
WebSocket	Protocole de communication bidirectionnelle en temps réel, utilisé par le backend pour écouter les événements émis par le smart contract.

Paiements et Devises

Sigle	Définition
EUR	Euro — devise de référence pour les transferts impliquant des banques de la zone euro (ex. BNP Paribas, Deutsche Bank).
FX	Foreign Exchange — change de devises ; désigne les opérations de conversion effectuées lors d'un transfert interbancaire.
SWIFT	Society for Worldwide Interbank Financial Telecommunication — réseau de messagerie interbancaire traditionnel utilisé comme référence de comparaison dans l'étude de marché (chapitre 1).
TND	Dinar tunisien — devise nationale tunisienne, devise de référence de la plupart des banques partenaires du système.
USD	United States Dollar — dollar américain, l'une des trois devises supportées par la plateforme avec l'EUR et le TND.



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454